

Introduction to Artificial Intelligence

Classical Planning and PDDL

Matteo Acclavio

07/05/2026

Plan

Some time ago:

- Factorizing the environment
- logic-based agent design
- Search as theorem proving

Today:

- Classical planning
- PDDL syntax and structure

Recap on Logic

Components of a logic $\mathcal{L}$:

- Syntax: how to write formulas
- Semantics: what formulas mean

Useful to have:

- Inference rules: how to derive new formulas from old ones
- Proof system S : a collection of inference rules that is sound and complete with respect to the semantics

$$\vdash_S F \iff \models_{\mathcal{L}} S$$

Propositional Logic

Components of the logic:

- Syntax: propositional formulas

$$A, B := \top \mid \perp \mid p \mid \neg A \mid A \wedge B \mid A \vee B \mid A \rightarrow B \quad p \in \mathbb{A}$$

- Semantics: truth values

$$\mathfrak{M} : \mathbb{A} \rightarrow \{\top, \perp\}$$

[it suffices to assign values to propositional variables, and truth tables for connectives]

Useful to have:

- Resolution:

$$\text{res} \frac{[A_1, \dots, A_n, C][\neg C, B_1, \dots, B_m]}{[A_1, \dots, A_n, B_1, \dots, B_m]}$$

... + convert to CNF + handle "duplications"

First-Order Logic

Components of the logic:

- Syntax: first-order formulas

$$A, B = P(t_1, \dots, t_n) \mid \neg A \mid A \wedge B \mid A \vee B \mid \forall x. A \mid \exists x. A \\ t, t_1, \dots, t_n = x \mid c \mid f(t_1, \dots, t_n) \quad x \in \mathbb{X}, c \in \mathbb{C}, f \in \mathbb{F}$$

- Semantics: truth values

$$\mathfrak{M} : \mathbb{A} \rightarrow \langle \mathbb{D}, \mathcal{I} \rangle$$

where $\mathbb{D}$ is a non-empty domain of discourse and $\mathcal{I}$ is an interpretation function that assigns meanings to constants, functions, and predicates.

Useful to have:

- Resolution:

$$\text{res} \frac{[A_1, \dots, A_n, C(t_1, \dots, t_k)][\neg C(t'_1, \dots, t'_k), B_1, \dots, B_m]}{\sigma([A_1, \dots, A_n, B_1, \dots, B_m])} \sigma = \text{mgu}(\dots)$$

... + convert to CNF + handle "duplications" + unification and substitution

Assumptions we are going to use in First-Order Logic

- Unique names: different constants refer to different objects

Assumptions we are going to use in First-Order Logic

- Unique names: different constants refer to different objects
- Closed-world: what is not known to be true is false

Assumptions we are going to use in First-Order Logic

- Unique names: different constants refer to different objects
- Closed-world: what is not known to be true is false
- Domain closure: the only objects in the domain are those named by constants

Assumptions we are going to use in First-Order Logic

- Unique names: different constants refer to different objects
- Closed-world: what is not known to be true is false
- Domain closure: the only objects in the domain are those named by constants

Database semantics

How do we use this in practice?

Classical Planning

Classical Planning Assumptions

A classical planning problem assumes:

- Fully observable world state
- Deterministic action effects
- Static world model while planning
- Discrete states and actions
- Single-agent setting

Planning asks for an action sequence that transforms the initial state into a state satisfying the goal.

Why Planning? Representative Applications

Planning models can describe very different domains:

- Puzzle solving (Sokoban, Sliding Tile Puzzle)
- Robotics and autonomy (Mars rover activity planning)
- Industrial logistics (greenhouse and warehouse routing)

Why Planning? Representative Applications

Planning models can describe very different domains:

- Puzzle solving (Sokoban, Sliding Tile Puzzle)
- Robotics and autonomy (Mars rover activity planning)
- Industrial logistics (greenhouse and warehouse routing)

Key idea: **domain-independent planning**.

We encode the model once and reuse general-purpose planners.

Core Components

- **States:** conjunctions of ground facts
- **Actions:** preconditions + effects
- **Initial state:** facts true at the start
- **Goal:** facts that must hold at the end
- **Plan:** action sequence $\langle a_1, \dots, a_n \rangle$

Core Components

- **States:** conjunctions of ground facts
- **Actions:** preconditions + effects
- **Initial state:** facts true at the start
- **Goal:** facts that must hold at the end
- **Plan:** action sequence $\langle a_1, \dots, a_n \rangle$

Closed-world assumption: facts not listed in a state are false.

Formal Problem Definition

A classical planning problem is

$$P = \langle V, A, s_I, g \rangle$$

where:

- V : finite set of propositional variables (facts)
- A : finite set of actions
- $s_I \subseteq V$: initial state
- $g \subseteq V$: goal condition

The state space is $S = 2^V$, and goal states are $\{s \in S \mid s \text{ satisfies } g\}$.

State Transition Semantics

For each action $a \in A$ we define:

- precondition set $pre(a)$
- add list $add(a)$
- delete list $del(a)$

State Transition Semantics

For each action $a \in A$ we define:

- precondition set $pre(a)$
- add list $add(a)$
- delete list $del(a)$

Transition function from a state s and action a :

$$\gamma(s, a) = \begin{cases} s' = (s \setminus del(a)) \cup add(a) & \text{if } pre(a) \subseteq s \\ \text{undefined} & \text{otherwise} \end{cases}$$

State Transition Semantics

For each action $a \in A$ we define:

- precondition set $pre(a)$
- add list $add(a)$
- delete list $del(a)$

Transition function from a state s and action a :

$$\gamma(s, a) = \begin{cases} s' = (s \setminus del(a)) \cup add(a) & \text{if } pre(a) \subseteq s \\ \text{undefined} & \text{otherwise} \end{cases}$$

Definition: Valid Plan

A plan $\langle a_1, \dots, a_n \rangle$ is valid if

$$\gamma((\dots \gamma(\gamma(s_I, a_1), a_2) \dots), a_n)$$

is defined and satisfies g .

Logically...

We can represent states as conjunctions of facts, and actions as implications:

- State s : $F_1 \wedge F_2 \wedge \dots \wedge F_k$
- Action a : $pre(a) \rightarrow (add(a) \wedge \neg del(a))$
- ... plus additional implications to capture state updates and frame axioms

Logically...

We can represent states as conjunctions of facts, and actions as implications:

- State s : $F_1 \wedge F_2 \wedge \dots \wedge F_k$
- Action a : $pre(a) \rightarrow (add(a) \wedge \neg del(a))$
- ... plus additional implications to capture state updates and frame axioms

Planning reduces to proving that the goal can be derived from the initial state and action implications.

$$\Gamma_{\text{Start}}, \text{Actions} \vdash \Gamma_{\text{Goal}} \quad \text{or} \quad \Gamma_{\text{Start}} \vdash_{\text{Actions}} \Gamma_{\text{Goal}}$$

Search for Plans

- Forward search: start from s_I and apply actions to reach a state satisfying g
- Backward search: start from g and find actions that can achieve it, then recursively find actions to achieve their preconditions

Tip: we can look at the backward search as goal-oriented proof search in a logic with the inference rule:

$$\text{Act} \frac{\Gamma}{(\Gamma \setminus \text{del}(a)), \text{add}(a)} \Gamma \supseteq \text{pre}(a)$$

- SATPlan: encode the problem as a propositional satisfiability problem and use a SAT solver
- Heuristic search: use heuristics to estimate distance to goal and guide search

Example: Blocks World

Blocks World: Intuition

Objects are blocks that can be stacked into towers.

Typical constraints:

- only blocks on top of a stuck (clear) can be moved
- one block can be held at a time
- a block can be moved to table or onto another clear block

This domain is a canonical benchmark for planning complexity and heuristics.

Blocks World Problem Example (3 blocks)

Initial state:

$OnTable(a) \wedge OnTable(b) \wedge OnTable(c) \wedge$
 $Clear(a) \wedge Clear(b) \wedge Clear(c) \wedge HandEmpty$

Goal:

$On(a, b) \wedge On(b, c)$

Actions:

Action(*Pickup*(x),
Precond: $OnTable(x) \wedge Clear(x) \wedge HandEmpty$,
Effect: $Holding(x) \wedge \neg OnTable(x) \wedge \neg Clear(x)$
 $\wedge \neg HandEmpty$)

Action(*Stack*(x, y),
Precond: $Holding(x) \wedge Clear(y)$,
Effect: $On(x, y) \wedge Clear(x) \wedge HandEmpty \wedge$
 $\neg Holding(x) \wedge \neg Clear(y)$)

Valid plan: $\langle Pickup(b), Stack(b, c), Pickup(a), Stack(a, b) \rangle$

Example: Gripper Robot

Task Description

Gripper world:

- Rooms: roomA, roomB
- Balls: ball1, ball2, ball3, ball4
- Robot with two grippers: left, right

Initial state: robot and all balls in roomA, both grippers free.

Goal: all balls are in roomB.

Objects and Predicate Vocabulary

Objects:

- Rooms: roomA, roomB
- Balls: ball1, ball2, ball3, ball4
- Grippers: left, right

Predicates:

Room(x), Ball(x), Gripper(x), RobotAt(r), At(b, r), Free(g), Carry(g, b)

Initial State and Goal

Initial state:

$$\begin{aligned} & \text{Room}(\text{roomA}) \wedge \text{Room}(\text{roomB}) \\ \wedge & \text{Ball}(\text{ball1}) \wedge \text{Ball}(\text{ball2}) \wedge \text{Ball}(\text{ball3}) \wedge \text{Ball}(\text{ball4}) \\ \wedge & \text{RobotAt}(\text{roomA}) \wedge \text{Free}(\text{left}) \wedge \text{Free}(\text{right}) \\ \wedge & \text{At}(\text{ball1}, \text{roomA}) \wedge \text{At}(\text{ball2}, \text{roomA}) \\ \wedge & \text{At}(\text{ball3}, \text{roomA}) \wedge \text{At}(\text{ball4}, \text{roomA}) \end{aligned}$$

Goal:

$$\text{At}(\text{ball1}, \text{roomB}) \wedge \text{At}(\text{ball2}, \text{roomB}) \wedge \text{At}(\text{ball3}, \text{roomB}) \wedge \text{At}(\text{ball4}, \text{roomB})$$

Action Schemas

Action(*Move*(x, y),
Precond: $Room(x) \wedge Room(y) \wedge RobotAt(x)$,
Effect: $RobotAt(y) \wedge \neg RobotAt(x)$)

Action(*PickUp*(b, r, g),
Precond: $Ball(b) \wedge Room(r) \wedge Gripper(g) \wedge$
 $At(b, r) \wedge RobotAt(r) \wedge Free(g)$,
Effect: $Carry(g, b) \wedge \neg At(b, r) \wedge \neg Free(g)$)

Action(*Drop*(b, r, g),
Precond: $Ball(b) \wedge Room(r) \wedge Gripper(g) \wedge$
 $Carry(g, b) \wedge RobotAt(r)$,
Effect: $At(b, r) \wedge Free(g) \wedge \neg Carry(g, b)$)

More Planning

Hierarchical Task Network (HTN) planning extends classical planning by allowing tasks to be decomposed into subtasks, enabling more structured and efficient planning.

Hierarchical Task Network (HTN) planning extends classical planning by allowing tasks to be decomposed into subtasks, enabling more structured and efficient planning.

Non-deterministic domains allow actions to have multiple possible outcomes, requiring plans that can handle all contingencies and ensure goal achievement under uncertainty.

Hierarchical Task Network (HTN) planning extends classical planning by allowing tasks to be decomposed into subtasks, enabling more structured and efficient planning.

Non-deterministic domains allow actions to have multiple possible outcomes, requiring plans that can handle all contingencies and ensure goal achievement under uncertainty.

- **Sensorless planning** considers scenarios where the agent has no information about the initial state, requiring plans that succeed regardless of the actual state. Here we need to use the **open-world assumption** instead of the closed-world one.

Hierarchical Task Network (HTN) planning extends classical planning by allowing tasks to be decomposed into subtasks, enabling more structured and efficient planning.

Non-deterministic domains allow actions to have multiple possible outcomes, requiring plans that can handle all contingencies and ensure goal achievement under uncertainty.

- **Sensorless planning** considers scenarios where the agent has no information about the initial state, requiring plans that succeed regardless of the actual state. Here we need to use the **open-world assumption** instead of the closed-world one.
- **Probabilistic planning** models uncertainty in action outcomes, enabling decision-making under uncertainty and the generation of robust plans.

Hierarchical Task Network (HTN) planning extends classical planning by allowing tasks to be decomposed into subtasks, enabling more structured and efficient planning.

Non-deterministic domains allow actions to have multiple possible outcomes, requiring plans that can handle all contingencies and ensure goal achievement under uncertainty.

- **Sensorless planning** considers scenarios where the agent has no information about the initial state, requiring plans that succeed regardless of the actual state. Here we need to use the **open-world assumption** instead of the closed-world one.
- **Probabilistic planning** models uncertainty in action outcomes, enabling decision-making under uncertainty and the generation of robust plans.

Temporal planning incorporates time constraints and durations for actions, allowing for scheduling and concurrent actions.

Summary

- Classical planning is a methodology to structure logic-based agent design
- Planners perform systematic searches for action sequences from init to goal
- Good models are precise, minimal, and easy to validate
- Planning can be adapted to hierarchical, temporal, and probabilistic settings

Reference: Russell & Norvig, Chapter 11.1, 11.2, 11.4